

# Blockchain Simulation Project: Technical Documentation

## 1 Project Overview

This project implements a comprehensive blockchain simulation system in Python, designed to demonstrate core blockchain concepts including distributed consensus, cryptographic security, transaction validation, and peer-to-peer networking. The system features multiple node types that work together to form a functional blockchain network.

## 2 System Architecture

### 2.1 Core Components

The system consists of four main modules that work together to simulate blockchain functionality:

- **Blockchain Core (blockchain.py):** Manages the chain state, transaction validation, and mining operations
- **Block Structure (block.py):** Defines the block data structure and Merkle tree implementation
- **Transaction System (transactions.py):** Handles cryptographic transactions, signatures, and UTXO management
- **Network Layer (node.py, protocols.py):** Implements P2P networking and node communication protocols

### 2.2 Node Types

The simulation features three specialized node types:

#### *2.2.1 Bootstrapper Node*

**Purpose:** Network discovery and peer coordination

**Functionality:**

- Maintains registry of active peers
- Distributes peer lists to new nodes
- Acts as initial contact point for network entry

### *2.2.2 Core Node*

**Purpose:** Blockchain validation and propagation

**Functionality:**

- Validates transactions and blocks
- Maintains UTXO pool
- Propagates valid transactions/blocks
- Handles chain synchronization

### *2.2.3 Miner Node*

**Purpose:** Block creation and consensus

**Functionality:**

- Mines new blocks using Proof-of-Work
- Manages transaction pool
- Competes in mining race
- Handles fork detection and resolution

## **3 Technical Implementation Details**

### **3.1 Cryptographic Foundations**

#### *3.1.1 Digital Signatures*

- Uses ECDSA with SECP256k1 curve (Bitcoin standard)
- Public keys hashed using SHA-256 for address generation
- Transaction signing prevents tampering and ensures authenticity

#### *3.1.2 Hash Functions*

SHA-256 is used for:

- Block hashing
- Transaction IDs
- Merkle tree construction
- Proof-of-Work difficulty verification

## 3.2 Blockchain Data Structures

### 3.2.1 Block Structure

```
class Block:
    index: int # Block height
    transactions: List[Transaction] # Contained transactions
    timestamp: float # Creation time
    previous_hash: str # Parent block hash
    nonce: int # Proof-of-Work solution
    merkle_tree: str # Merkle root hash
```

### 3.2.2 Transaction Model (UTXO-based)

- **Inputs:** Reference to previous transaction outputs
- **Outputs:** New UTXOs created by the transaction
- **Validation:** Checks signatures, UTXO existence, and double-spending

## 3.3 Consensus Mechanism

### 3.3.1 Proof-of-Work Implementation

- Difficulty target: Hash must start with "0000"
- Mining process: Incremental nonce search
- Chain selection: Longest valid chain rule

### 3.3.2 Fork Resolution

- Automatic chain validation across nodes
- Conflict detection through hash comparison
- Chain replacement when longer valid chain received
- **Test Coverage:** Simulated concurrent mining by multiple miners producing forks, demonstrating resolution strategies

## 3.4 Network Protocol

### 3.4.1 Message Types

|              |                                       |
|--------------|---------------------------------------|
| registration | → Node registration with bootstrapper |
| get_peers    | → Request peer list                   |
| peers        | → Respond with peer list              |
| get_chain    | → Request full blockchain             |
| new_chain    | → Send blockchain data                |
| transaction  | → Broadcast new transaction           |
| block        | → Broadcast new block                 |

### 3.4.2 *Communication Protocol*

- Fixed-length headers for message type identification
- Length-prefixed payloads using pickle serialization
- Asynchronous message handling with worker threads
- **Test Coverage:** Verified through chain synchronization tests between nodes and peer discovery via bootstrapper

## 4 **Key Algorithms and Mechanisms**

### 4.1 **Merkle Tree Implementation**

#### 4.1.1 *Construction Algorithm*

- Hash all transactions to create leaf nodes
- Pair-wise hashing of nodes until single root remains
- Handle odd-numbered levels by duplicating last hash

#### 4.1.2 *Proof Generation*

- Creates inclusion proofs for transaction verification
- Efficient verification without full tree reconstruction

### 4.2 **Transaction Validation Pipeline**

1. UTXO Existence Check: Verify referenced outputs exist
2. Signature Verification: Validate all input signatures
3. Double-Spending Prevention: Check for duplicate inputs
4. Value Conservation: Ensure inputs  $\geq$  outputs
5. Pubkey Hash Matching: Verify spending authority

### 4.3 **Block Validation Process**

- Structural Integrity: Verify block hash meets difficulty
- Transaction Validity: Validate all contained transactions
- Chain Linkage: Check previous hash matches parent block
- Merkle Root Verification: Confirm correct transaction commitment

## 5 Simulation Features

### 5.1 Testing Framework

The project includes a comprehensive `pytest`-based test suite covering:

- **Transaction Validation:** Valid and invalid transactions, signature checks, and UTXO spending
- **Block Validation:** Valid/invalid blocks, structural checks, and linkage
- **Chain Validation:** Sequential mining of multiple blocks, UTXO updates, and chain integrity
- **Network Synchronization:** Core nodes requesting and receiving blockchain state from peers
- **Bootstrapper Discovery:** New nodes obtaining peer lists from the bootstrapper
- **Transaction Propagation:** Broadcasting transactions across multiple nodes
- **Mining Pipeline:** Miner event signaling, transaction pool consumption, and block propagation
- **Fork Detection:** Multiple miners racing to mine blocks concurrently, creating and resolving forks

### 5.2 Concurrency Handling

- Thread-safe data structures for chain state
- Event-driven mining activation (triggered when enough transactions are in the pool)
- Asynchronous message processing across peers
- Lock-based resource protection ensuring mining and pool updates are safe under concurrency

## 6 Educational Objectives Achieved

### 6.1 Blockchain Fundamentals Demonstrated

- **Decentralization:** P2P network without central authority
- **Immutability:** Cryptographic chaining of blocks
- **Consensus:** Distributed agreement on chain state
- **Security:** Cryptographic protection against tampering

## **6.2 Technical Concepts Implemented**

- Cryptographic Hashing: Data integrity verification
- Digital Signatures: Authentication and non-repudiation
- Merkle Trees: Efficient data verification
- UTXO Model: Bitcoin-style transaction handling
- Proof-of-Work: Sybil attack resistance

## **6.3 Distributed Systems Principles**

- Network Protocols: Custom P2P communication
- State Synchronization: Chain replication across nodes
- Fault Tolerance: Handling network partitions
- Conflict Resolution: Fork detection and chain selection

# **7 Project Limitations and Learning Outcomes**

## **7.1 Simulation Constraints**

- Simplified difficulty adjustment
- Basic transaction fee model
- Limited scalability optimizations
- Simplified network topology

## **7.2 Key Learning Outcomes**

- Deep understanding of blockchain architecture
- Hands-on experience with cryptographic primitives
- Distributed systems design and implementation
- Consensus algorithm practical implementation
- Network protocol design and testing

# **Conclusion**

This project successfully demonstrates the core principles of blockchain technology through a functional simulation, validated by an extensive test suite. The modular design allows for easy extension and experimentation with different consensus mechanisms or cryptographic approaches, while the testing framework ensures robustness across real-world scenarios such as transaction propagation, peer discovery, and fork resolution.

*Note: This documentation focuses on the technical implementation aspects. The accompanying test files provide practical examples of system behavior under various scenarios.*